

Politechnika Rzeszowska im. Ignacego Łukasiewicza

Wydział Matematyki i Fizyki Stosowanej



WYDZIAŁ
MATEMATYKI
I FIZYKI STOSOWANEJ
POLITECHNIKI RZESZOWSKIEJ

System zarządzania agencją mieszkaniową

Mateusz Gałda, Paweł Górski

FSO-DI

Rzeszów 2026

Spis treści

Wstęp i Cel projektu.....	3
Architektura Aplikacji	3
Technologie i struktura	3
Folder Backend	3
Folder Frontend	3
Baza danych	3
Funkcjonalności	3
Podsumowanie i wnioski.....	22

Wstęp i Cel projektu

Celem projektu było stworzenie działającej aplikacji internetowej, następnie zapewnienie jej zabezpieczeń. Projekt przedstawia działającą witrynę agencji nieruchomości w architekturze klient-serwer, z wykorzystaniem technologii PHP, MySQL oraz JS.

Architektura Aplikacji

Technologie i struktura

Folder Backend

Zawiera pliki kluczowe dla logiki strony zapewniające jej biznesowe działanie. Są tam takie pliki jak: api.php, auth.php oraz config.php.

Folder Frontend

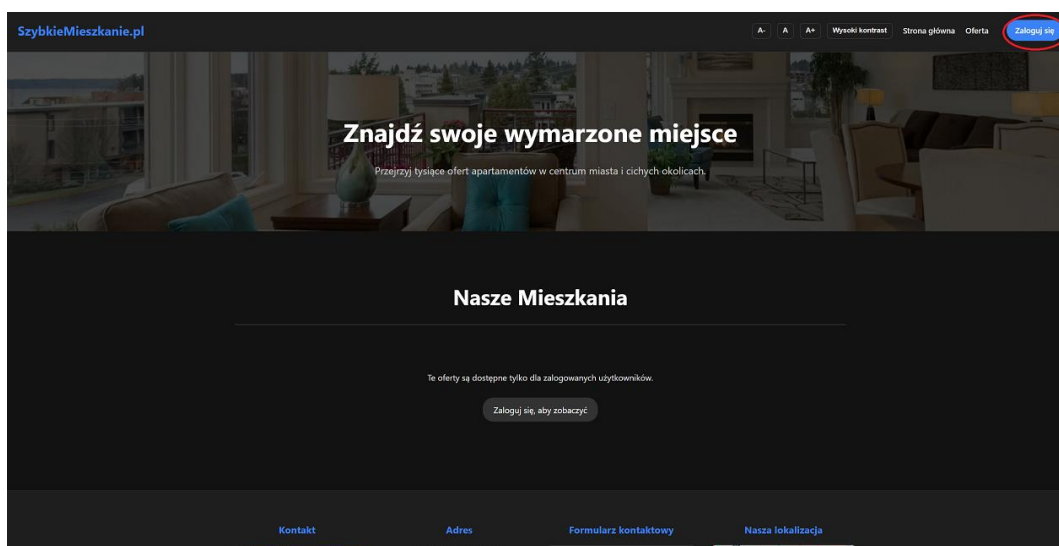
Zawiera pliki odpowiedzialne za wizualną prezentację strony między innymi app.js do dynamicznego pobierania danych z bazy, następnie style.css odpowiadające za oprawę wizualną.

Baza danych

Dodatkowo jest zamieszczony plik backup bazy danych, która jest połączona z witryną.

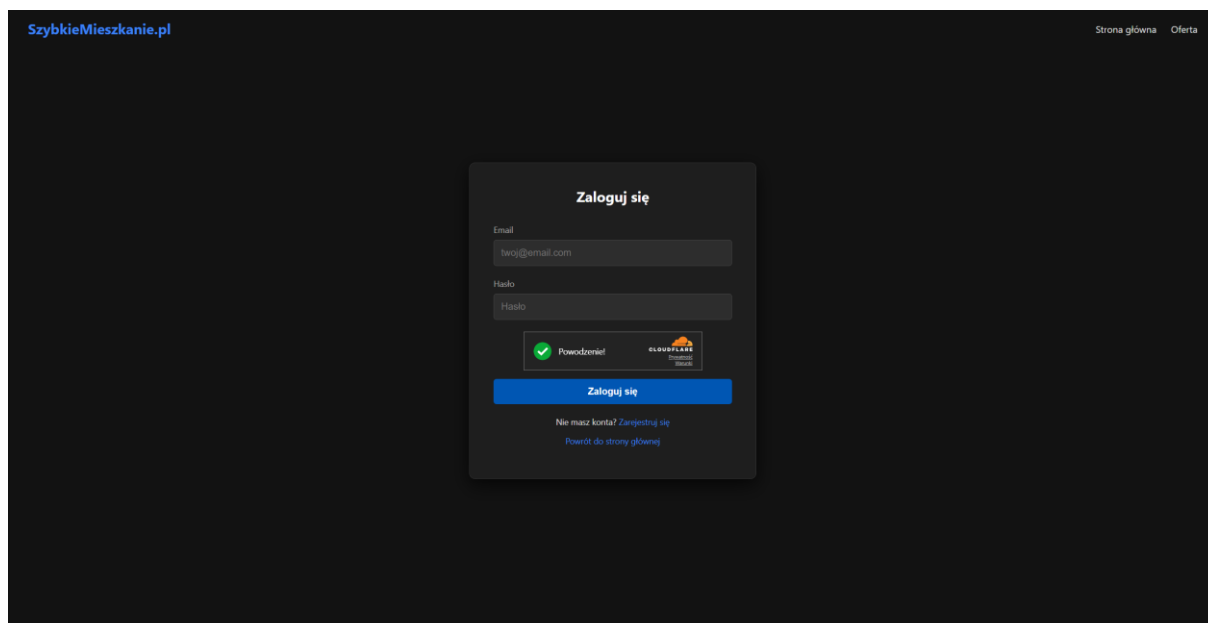
Funkcjonalności

- System logowania



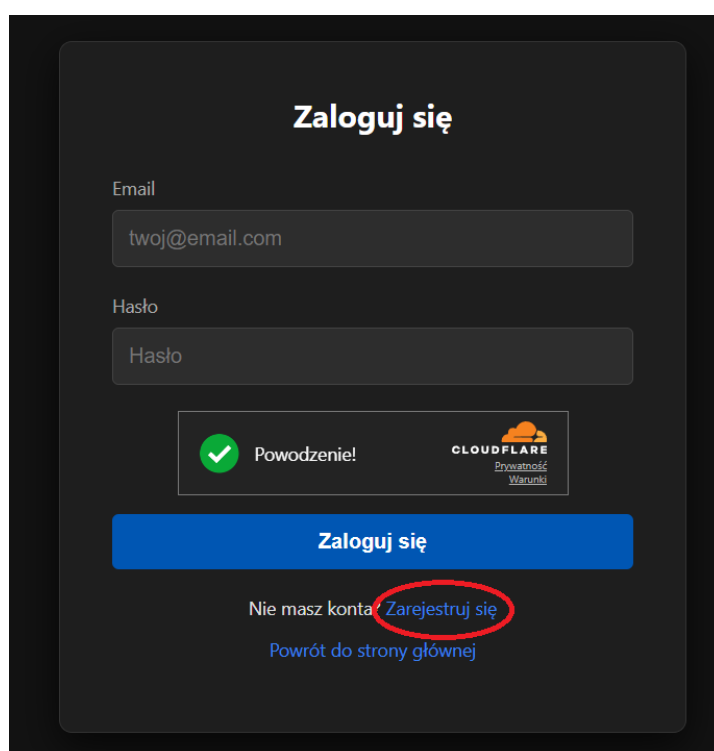
Rys. 1 Strona główna z zaznaczonym przyciskiem zaloguj się.

Na stronie głównej jest możliwość zalogowania się, będzie to niezbędne aby uzyskać dostęp do pełnych informacji i możliwości witryny, po kliknięciu go jesteśmy przekierowani do formularza logowania:



Rys. 2 Formularz logowania do strony

W tym miejscu należy podać mail oraz hasło. Jeżeli nie mamy jeszcze konta, możemy z poziomu formularza logowania przejść do formularza reejstracji:



Rys. 3 Przejście z formularza logowania do formularza rejestracji

Jeżeli podane dane będą błędne pojawia się komunikat o podaniu błędnych danych oraz ostrzeżenie o pozostałych próbach.

The screenshot shows a login form with the title "Zaloguj się". It has two input fields: "Email" with the value "mail@mail.com" and "Hasło" (Password) which is currently empty. Below the password field, there is a green checkmark icon and the text "Powodzenie!" (Success!). To the right of this is the Cloudflare logo and the text "Prywatność" (Privacy) and "Warunki" (Terms). Below these elements is a blue button labeled "Zaloguj się". Under the button, there is a red error message: "✗ Błędny email lub hasło Pozostałe próby: 2/3". At the bottom, there are two links: "Nie masz konta? Zarejestruj się" and "Powrót do strony głównej".

Rys. 4 Formularz logowania po podaniu błędnych danych

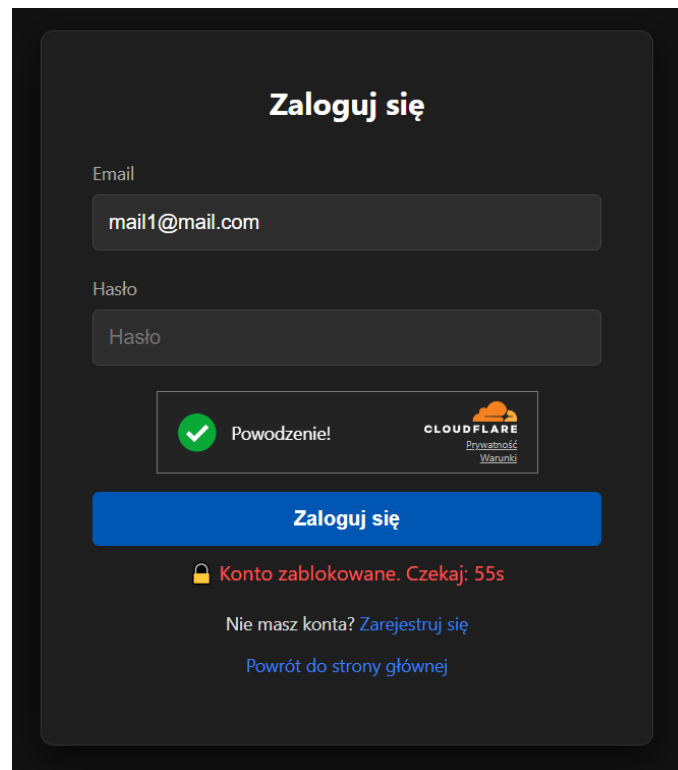
```
// BŁĄD LOGOWANIA - zwiększ licznik
lockData.attempts = (lockData.attempts || 0) + 1;
const attemptsLeft = 3 - lockData.attempts;

// Odliczanie
let secondsLeft = 60;
const blockInterval = setInterval(() => {
  secondsLeft--;
  messageBox.innerText = `🔒 Konto zablokowane.
Czekaj: ${secondsLeft}s`;

  if (secondsLeft <= 0) {
    clearInterval(blockInterval);
    messageBox.innerText = '✅ Możesz spróbować
zalogować się ponownie.';
    messageBox.style.color = '#4CAF50';
    submitBtn.disabled = false;
  }
}
```

Powyższy kod przedstawia logikę zliczanie błędów logowania oraz mechanizm nakładania blokady czasowej na 60s gdy będą 3 błędy.

W sytuacji kiedy podamy 3 razy błędne dane zostanie nam odebrany dostęp do tego formularza na 1 minutę:



Rys. 5 Formularz logowania z odebraniem dostępu

```
if (lockData.blockedUntil && now < lockData.blockedUntil) {  
  const secondsLeft = Math.ceil((lockData.blockedUntil - now) / 1000);  
  messageBox.innerText = `❌ Konto zablokowane na ${secondsLeft} sekund;  
  messageBox.style.color = '#ff5555';  
  submitBtn.disabled = true;  
  
  const countdownInterval = setInterval(() => {  
    const remaining = Math.ceil((lockData.blockedUntil - Date.now()) / 1000);  
    if (remaining <= 0) {  
      clearInterval(countdownInterval);  
      messageBox.innerText = "";  
      submitBtn.disabled = false;  
      localStorage.removeItem(lockKey);  
    } else {  
      messageBox.innerText = `❌ Konto zablokowane na ${remaining} sekund. Spróbuj później.`;  
    }  
  }, 1000);  
  return;  
}  
  
// BŁĄD LOGOWANIA - zwiększ licznik  
lockData.attempts = (lockData.attempts || 0) + 1;
```

```

const attemptsLeft = 3 - lockData.attempts;

if (lockData.attempts >= 3) {
  // BLOKADA NA 60 SEKUND
  lockData.blockedUntil = now + 60000;
  lockData.attempts = 0;
  localStorage.setItem(lockKey, JSON.stringify(lockData));

  messageBox.innerText = ' Zbyt wiele błędnych prób! Konto zablokowane na 60
sekund.';

  messageBox.style.color = '#ff5555';
  submitBtn.disabled = true;
  passwordInput.value = '';

  // Odliczanie
  let secondsLeft = 60;
  const blockInterval = setInterval(() => {
    secondsLeft--;
    messageBox.innerText = ` Konto zablokowane. Czekaj: ${secondsLeft}s`;

    if (secondsLeft <= 0) {
      clearInterval(blockInterval);
      messageBox.innerText = ' Możesz spróbować zalogować się ponownie.';
      messageBox.style.color = '#4CAF50';
      submitBtn.disabled = false;
    }
  }, 1000);

```

Powyższy kod przedstawia logikę działania wyświetlania pozostałych prób oraz zablokowania formularza na minutę.

Po podaniu poprawnych danych jesteśmy przekierowywane z powrotem na stronę główną, tym razem z dostępem do wszystkich informacji:

Zaloguj się

Email

Hasło

☒ Powodzenie!



CLOUDFLARE
Prywatność
Warunki

Zaloguj się

☒ Zalogowano! Przekierowywanie...

Nie masz konta? [Zarejestruj się](#)

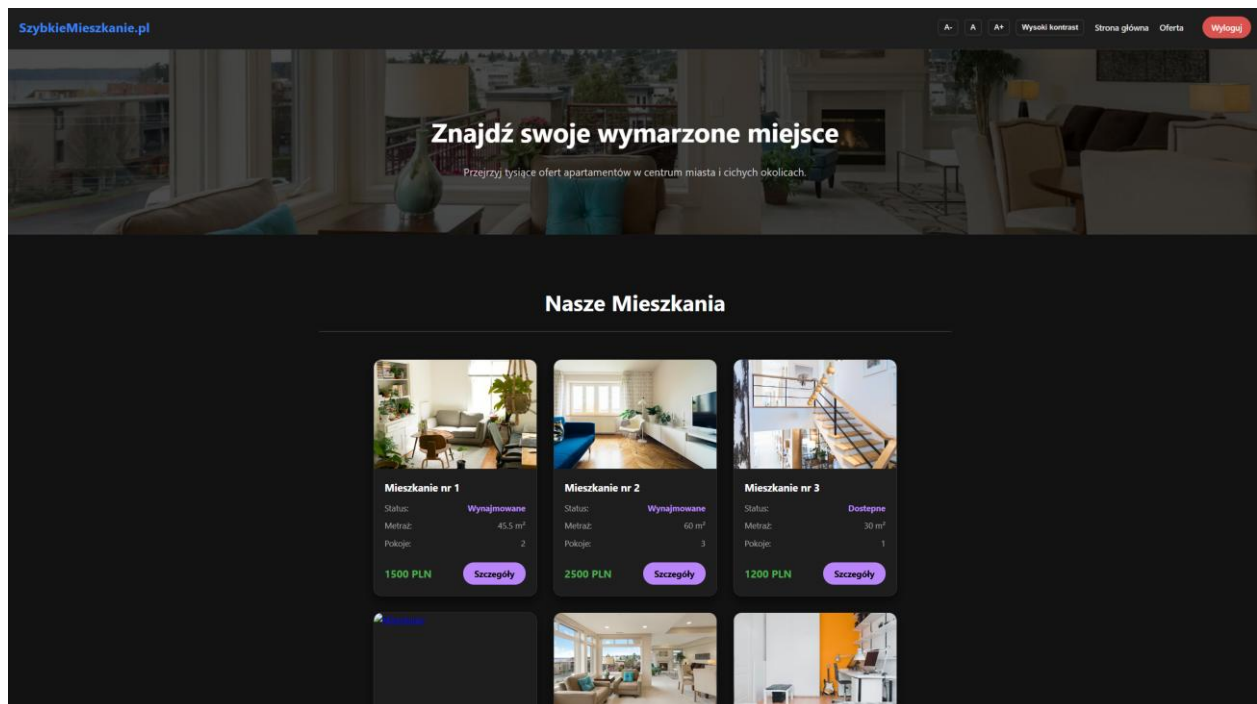
[Powrót do strony głównej](#)

Rys. 6 Formularz logowania po podaniu poprawnych danych

```
// LOGOWANIE OK - wyczyść blokadę
localStorage.removeItem(lockKey);
localStorage.setItem('userLoggedIn', 'true');

messageBox.innerText = '☑ Zalogowano!
Przekierowywanie...';
messageBox.style.color = '#4CAF50';
passwordInput.value = '';

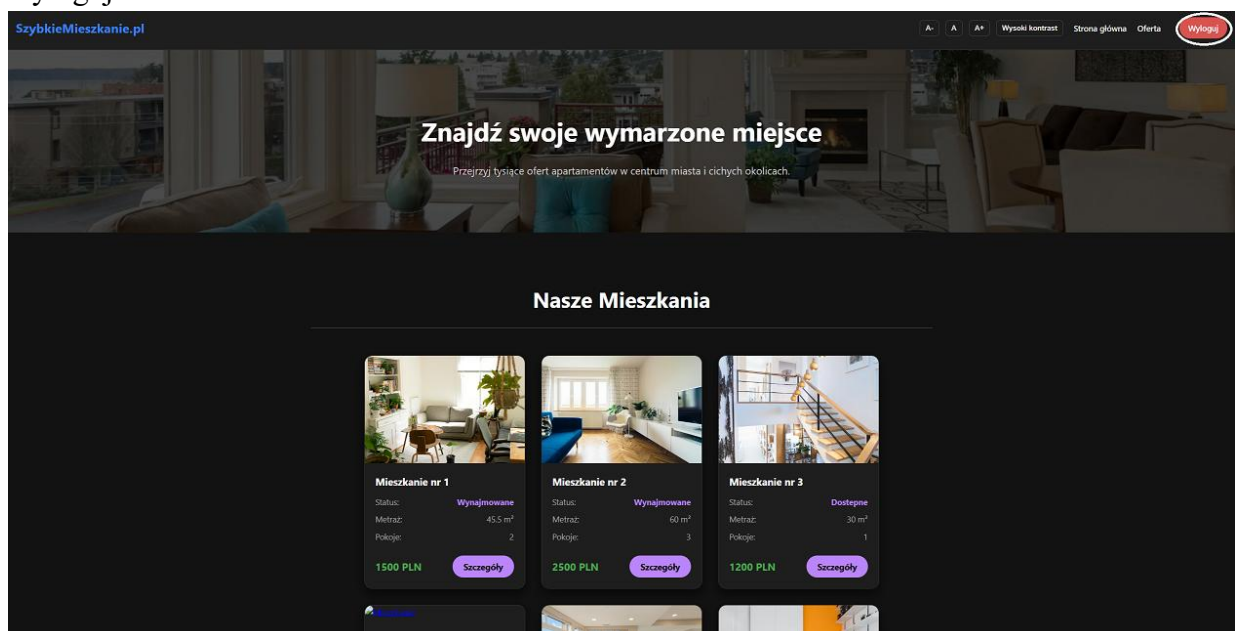
setTimeout(() => {
  window.location.href = '../public/index.html';
}, 1500);
```

Rys. 7 Strona główna po zalogowaniu

- System wylogowania

Po prawej stronie nagłówka po zalogowaniu przycisk zaloguj się, zostaje zmieniony na wyloguj:



Rys. 8 Strona główna z zaznaczonym przyciskiem wyloguj

```

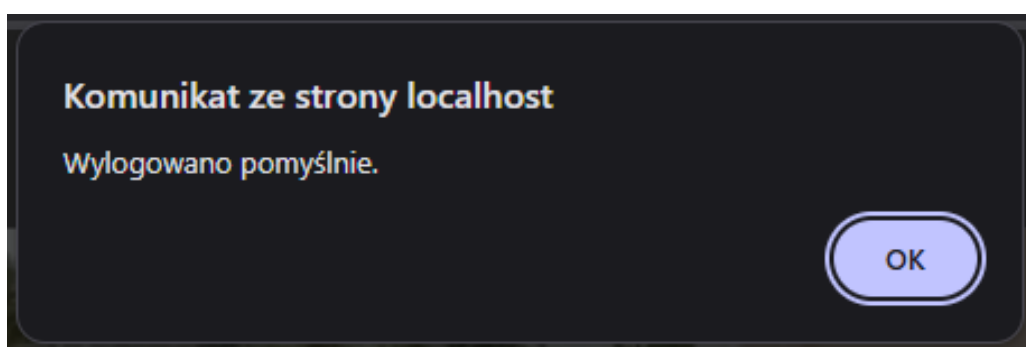
if (navLogoutBtn) {
    navLogoutBtn.addEventListener('click', function(e) {
        e.preventDefault();

        localStorage.removeItem('userLoggedIn');
        alert("Wylogowano pomyślnie.");

        window.location.reload();
    });
}

```

Po przyciśnięciu otrzymujemy komunikat ze strony o wylogowaniu, a następnie jest kasowana nasza sesja i tracimy dostęp do informacji na stronie:



Rys. 9 Komunikat ze strony przy wylogowywaniu

```

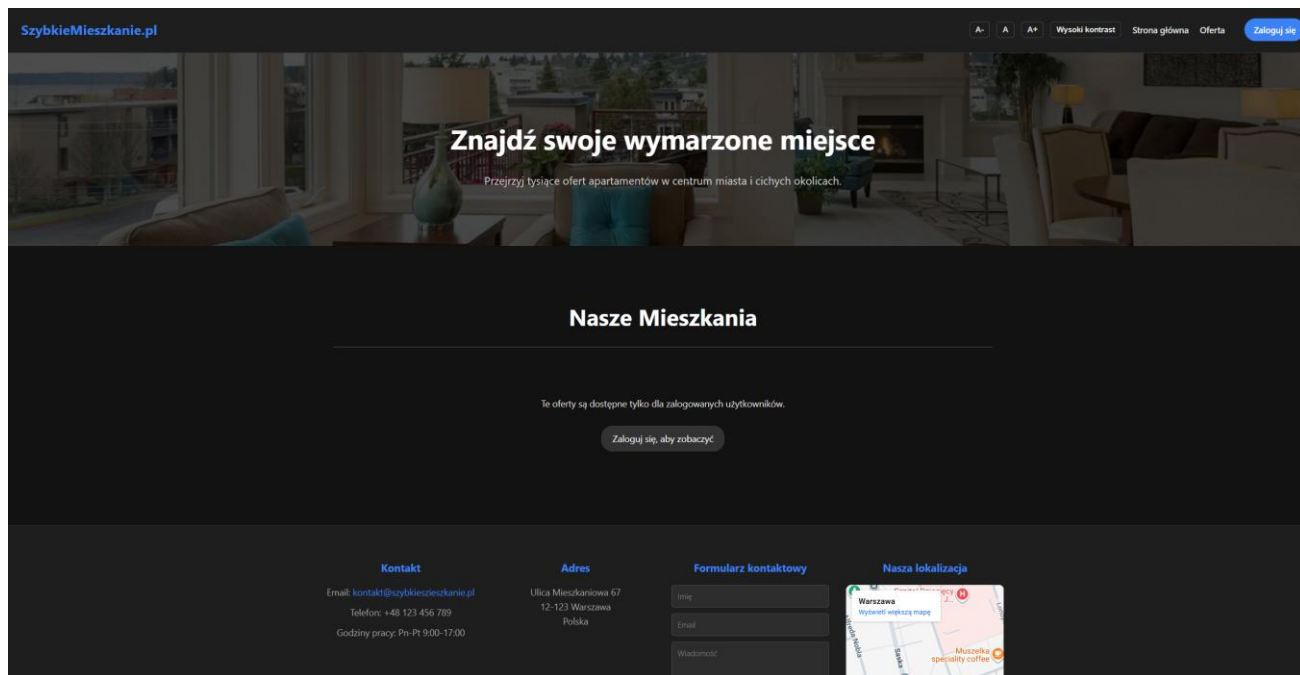
// Obsługa kliknięcia "Wyloguj"
if (navLogoutBtn) {
    navLogoutBtn.addEventListener('click', function(e) {
        e.preventDefault();

        localStorage.removeItem('userLoggedIn');
        alert("Wylogowano pomyślnie.");

        window.location.reload();
    });
}

```

Powyższy kod przedstawia działania komunikatu wylogowania.



Rys. 10 Strona główna po wylogowaniu

- System rejestracji

Formularz logowania zawiera 4 pola: Imię, nazwisko, adres mail oraz hasło, należy je uzupełnić poprawnymi danymi aby konto zostało utworzone

Rys. 11 Formularz logowania

```
// REJESTRACJA
const registerForm = document.getElementById('registerForm');

if (registerForm) {
    registerForm.addEventListener('submit', function(e) {
        e.preventDefault();

        const imieInput = document.getElementById('imie');
        const nazwiskoInput = document.getElementById('nazwisko');
        const emailInput = document.getElementById('reg-email');
        const passwordInput = document.getElementById('reg-password');
        const regMessageBox = document.getElementById('reg-message');
        const submitBtn =
registerForm.querySelector('button[type="submit"]');

        if (!imieInput || !nazwiskoInput || !emailInput || !passwordInput)
        {
            alert("Błąd: Formularz rejestracji nie ma wymaganych pól!");
            return;
        }

        const imieVal = imieInput.value.trim();
        const nazwiskoVal = nazwiskoInput.value.trim();
        const emailVal = emailInput.value.trim();
        const passVal = passwordInput.value;

        if (!imieVal || !nazwiskoVal || !emailVal || !passVal) {
            regMessageBox.innerText = '✗ Wypełnij wszystkie pola!';
            regMessageBox.style.color = '#ff5555';
            return;
        }

        if (!isValidEmail(emailVal)) {
            regMessageBox.innerText = '✗ Wpisz poprawny email!';
            regMessageBox.style.color = '#ff5555';
            return;
        }

        if (passVal.length < 6) {
            regMessageBox.innerText = '✗ Hasło musi mieć co najmniej 6
znaków!';

            regMessageBox.style.color = '#ff5555';
            return;
        }

        regMessageBox.innerText = 'Trwa rejestracja...';
        regMessageBox.style.color = '#666';
        submitBtn.disabled = true;
    });
}
```

```

const formData = new FormData(registerForm);
const turnstileResponse = formData.get('cf-turnstile-response');
fetch('../backend/api.php?action=register', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    imie: imieVal,
    nazwisko: nazwiskoVal,
    email: emailVal,
    password: passVal,
    'cf-turnstile-response': turnstileResponse
  })
})
.then(response => response.json())
.then(data => {
  if (data.status === 'success') {
    regMessageBox.innerHTML = '☑ ' + data.message;
    regMessageBox.style.color = '#4CAF50';
    registerForm.reset();

    setTimeout(() => {
      if (switchToLogin) switchToLogin.click();
    }, 2000);
  } else {
    regMessageBox.innerHTML = '✗ ' + data.message;
    regMessageBox.style.color = '#ff5555';
  }
  submitBtn.disabled = false;
})
.catch(error => {
  console.error('Błąd:', error);
  regMessageBox.innerHTML = '⚠ Błąd połączenia z serwerem!';
  regMessageBox.style.color = '#ff5555';
  submitBtn.disabled = false;
});
});
}

```

Przy podaniu w polu Email czegoś co nim nie jest, strona zwróci następujący komunikat:

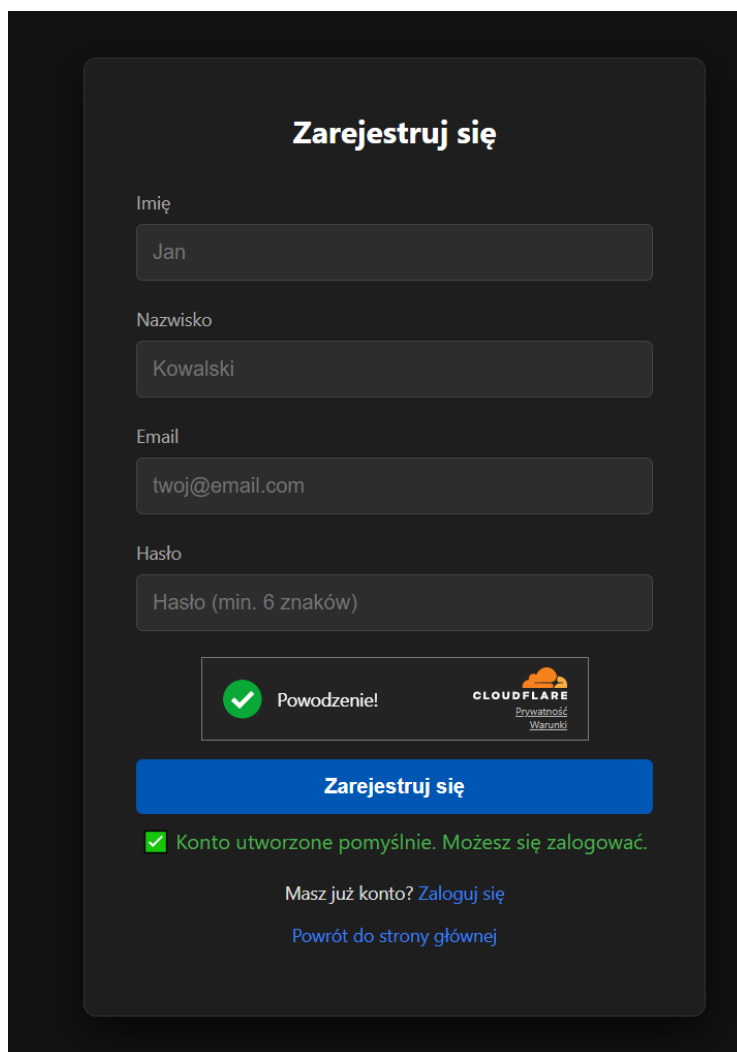
The screenshot shows a registration form with the title "Zarejestruj się". It contains three input fields: "Imię" (Name) with the value "test", "Nazwisko" (Surname) with the value "maila", and "Email" with the value "123123123123". Below the email field, a red error message box is displayed: "Uwzględnij znak „@” w adresie e-mail. W adresie „123123123123” brakuje znaku „@”." (Consider the "@" symbol in the email address. The address "123123123123" is missing the "@" symbol.). Below the error message is a password field with masked characters ".....". At the bottom of the form, there is a green checkmark icon and the text "Powodzenie!" (Success!), followed by the Cloudflare logo and the text "Przebieganie" (Progress). A blue button labeled "Zarejestruj się" is also present. Below the button, there are two links: "Masz już konto? Zaloguj się" (Do you have an account? Log in) and "Powrót do strony głównej" (Return to the main page).

Rys. 12 Komunikat strony na fałszywy mail

Wynika to z walidacji podawanych danych w formularzu:

```
function isValidEmail(email) {  
    const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
    return regex.test(email);  
}  
if (!isValidEmail(emailVal)) {  
    regMessageBox.innerText = '✗ Wpisz poprawny email!';  
    regMessageBox.style.color = '#ff5555';  
    return;  
}
```

Po podaniu poprawnych danych konto zostaje zapisane w bazie danych z zaszyfrowanym hasłem:



The image shows a registration form titled "Zarejestruj się" (Register) on a dark background. The form contains four input fields: "Imię" (Name) with the value "Jan", "Nazwisko" (Surname) with the value "Kowalski", "Email" with the value "twoj@email.com", and "Hasło" (Password) with the placeholder "Hasło (min. 6 znaków)". Below the fields is a green checkmark icon and the text "Powodzenie!" (Success!). To the right of this is the Cloudflare logo and the text "Przywróć Warunki" (Restore Terms). Below this is a blue button labeled "Zarejestruj się". At the bottom, there is a green checkmark icon and the text "Konto utworzone pomyślnie. Możesz się zalogować." (Account created successfully. You can log in.). Below this are two links: "Masz już konto? Zaloguj się" (Already have an account? Log in) and "Powrót do strony głównej" (Return to the main page).

Rys. 13 Komunikat strony na pomyślną rejestrację konta

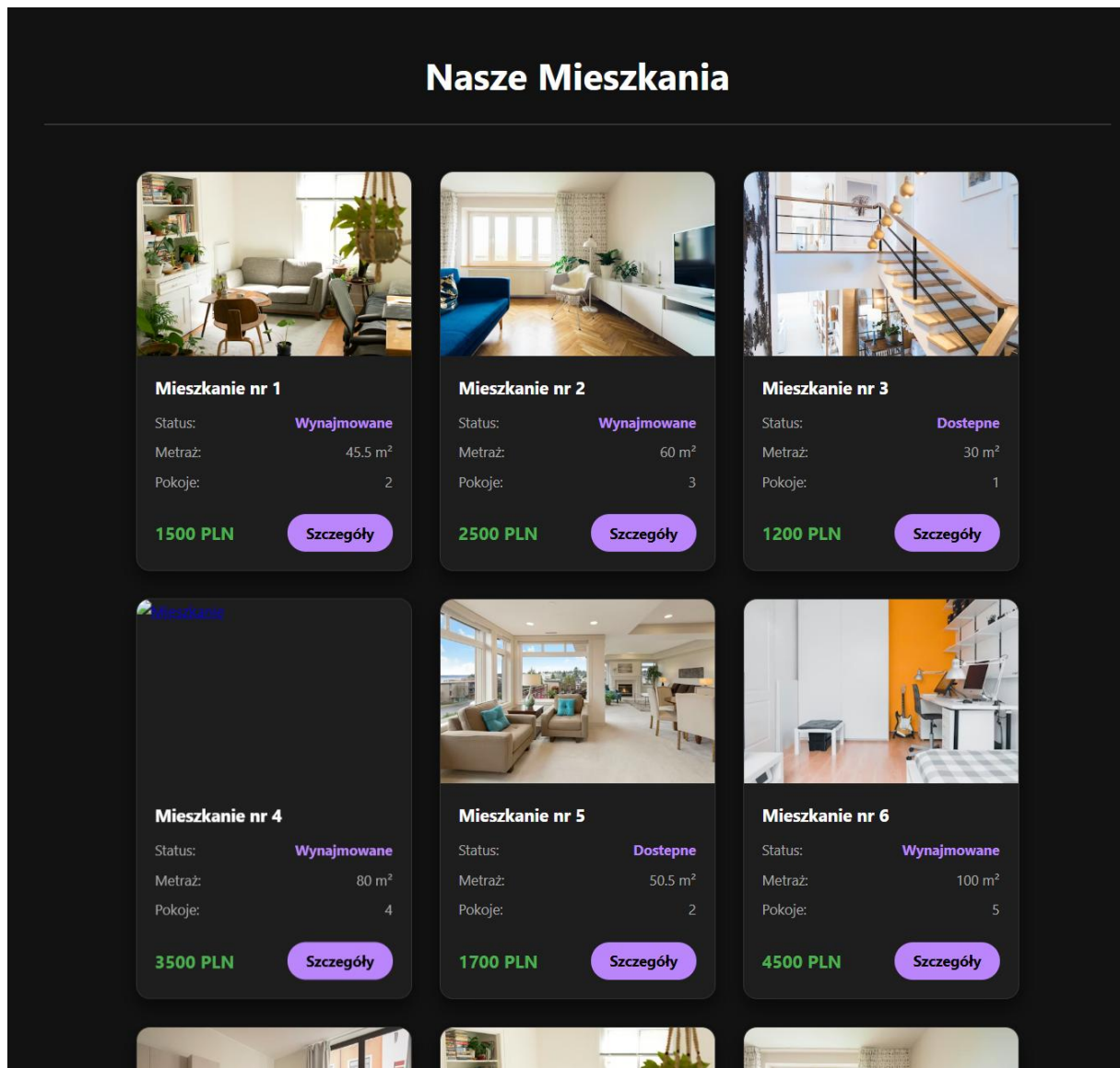
```
const turnstileResponse = formData.get('cf-turnstile-response');
fetch('../backend/api.php?action=register', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    imie: imieVal,
    nazwisko: nazwiskoVal,
    email: emailVal,
    password: passVal,
    'cf-turnstile-response': turnstileResponse
  })
})
.then(response => response.json())
.then(data => {
```

Powyższy kod przedstawia działanie komunikatów formularza rejestracji.

Rys. 14 Konta użytkowników ze strony bazy danych

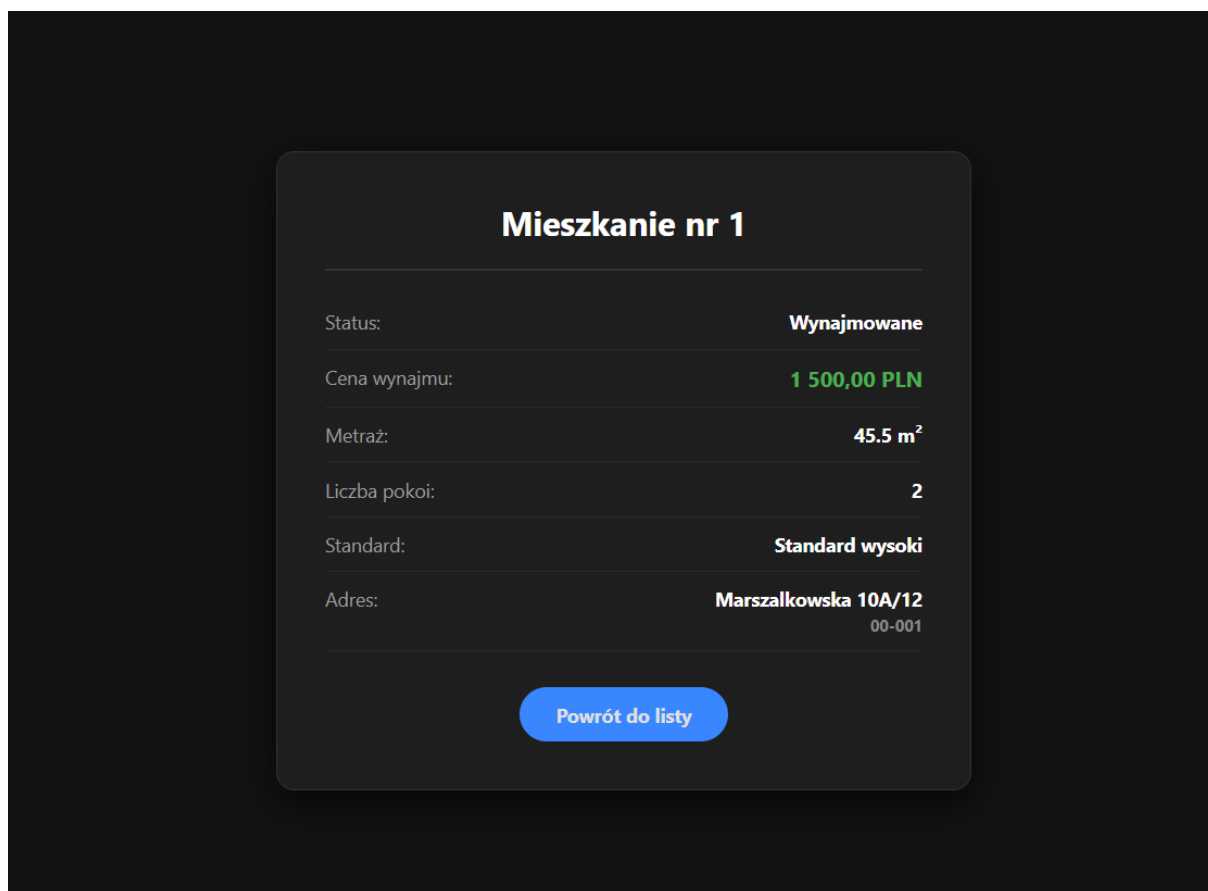
- Przegląd mieszkań

Zalogowany użytkownik otrzymuje dostęp do bazy mieszkań, którą może przeglądać na witrynie:



Rys. 15 Widok mieszkań na stronie dla zalogowanego użytkownika

Po kliknięciu na szczegóły, otrzymujemy szczegółowe informacje danego mieszkania:



Rys. 16 Szczegółowe informacje dla mieszkania nr 1

```
<?php
    $polaczenie = mysqli_connect('localhost', 'root', '', 'agencja_finalowa');

    if (!$polaczenie)
        die("Błąd połączenia z bazą: " . mysqli_connect_error());

    if (isset($_GET['id'])) {
        $id = intval($_GET['id']);

        $sql = "SELECT * FROM mieszkanie
                LEFT JOIN adres ON mieszkanie.adres_id_adresu =
adres.id_adresu
                LEFT JOIN standard_mieszkania ON
mieszkanie.standard_mieszkania_id_standardu = standard_mieszkania.id_standardu
                WHERE id_mieszkania = $id";

        $wynik = mysqli_query($polaczenie, $sql);
        $wiersz = mysqli_fetch_assoc($wynik);
    }
?>
```

Skrypt pobierający szczegóły o mieszkaniu z bazy danych.

```

<body>

    <div class="szczegoly-box">
        <?php if (isset($wiersz) && $wiersz): ?>

            <h1>Mieszkanie nr <?php echo $wiersz['id_mieszkania']; ?></h1>

            <div class="dane-wiersz">
                <span class="etykieta">Status:</span>
                <span class="wartosc"><?php echo $wiersz['status_mieszkania'];
?></span>
            </div>

            <div class="dane-wiersz">
                <span class="etykieta">Cena wynajmu:</span>
                <span class="wartosc cena"><?php echo
number_format($wiersz['cena_wynajmu'], 2, ',', ' '); ?> PLN</span>
            </div>

            <div class="dane-wiersz">
                <span class="etykieta">Metraż:</span>
                <span class="wartosc"><?php echo $wiersz['metraz_mieszkania'];
?> m²</span>
            </div>

            <div class="dane-wiersz">
                <span class="etykieta">Liczba pokoi:</span>
                <span class="wartosc"><?php echo $wiersz['liczba_pokoi'];
?></span>
            </div>

            <div class="dane-wiersz">
                <span class="etykieta">Standard:</span>
                <span class="wartosc">
                    <?php echo $wiersz['nazwa_standardu']; ?>
                </span>
            </div>

            <div class="dane-wiersz">
                <span class="etykieta">Adres:</span>
                <span class="wartosc">
                    <?php
                        echo $wiersz['ulica'] . ' ' . $wiersz['nr_domu'];
                        if(!empty($wiersz['nr_mieszkania'])) {
                            echo '/' . $wiersz['nr_mieszkania'];
                        }
                        echo '<br><small style="color:#888;">' .
$wiersz['kod_pocztowy'] . '</small>';
                    <?>
                </span>
            </div>

```

```

        </span>
    </div>

    <div class="przycisk-powrot">
        <a href="index.html" class="btn">Powrót do listy</a>
    </div>

    <?php else: ?>
        <h1 style="border:none;">Brak danych</h1>
        <p style="text-align:center;">Nie znaleziono mieszkania o takim
ID.</p>
        <div class="przycisk-powrot">
            <a href="index.html" class="btn">Wróć</a>
        </div>
    <?php endif; ?>
</div>

<?php mysqli_close($polaczenie); ?>

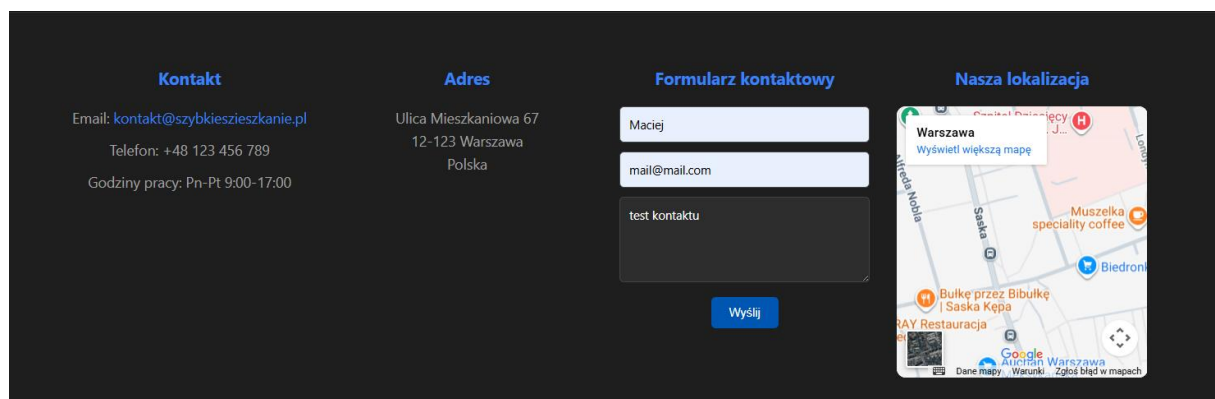
</body>

```

Powyższy od przedstawia strukturę wyświetlania szczegółów danego mieszkania.

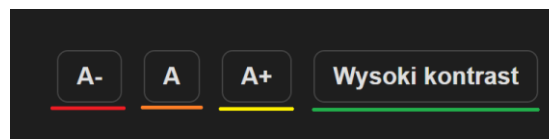
- Kontakt z administracją

W stopce strony zamieszczone są informacje o otwarciu biura, adres, formularz kontaktowy, oraz interaktywna mapa z siedzibą.



Rys. 17 Dane kontaktowe w stopce

- Funkcje dla osób niepełnosprawnych



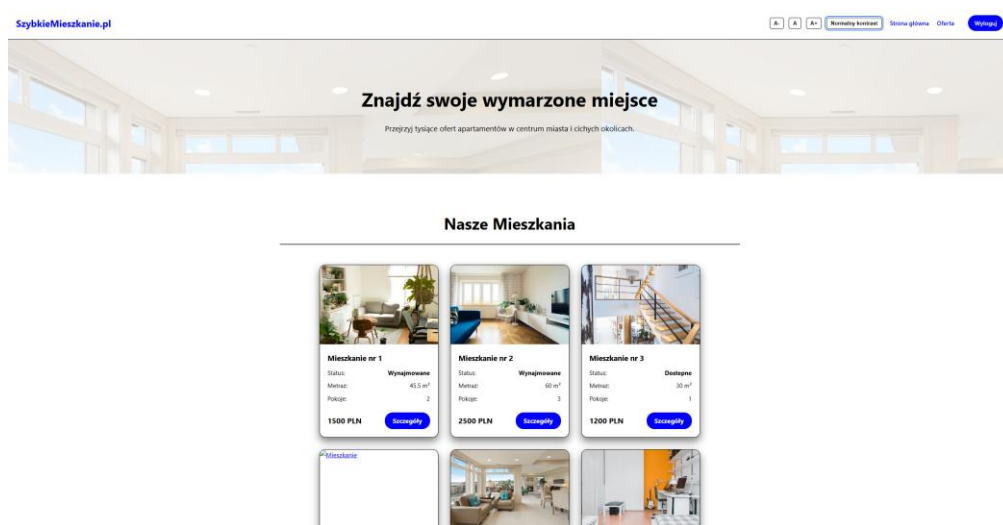
Rys. 18 Panel dla osób niepełnosprawnych

Przycisk zaznaczony na czerwono umożliwia zmniejszenie czcionki

Przycisk zaznaczony na pomarańczowo przywraca domyślny rozmiar czcionki

Przycisk zaznaczony na żółto zwiększa czcionkę

Przycisk zaznaczony na zielono zwiększa kontrast kolorów na stronie



Rys. 19 Strona główna z zwiększonym kontrastem

Dodatkowa opcja dla osób niepełnosprawnych to sterowania witryną za pomocą samej klawiatury, z funkcji dla osób niepełnosprawnych można korzystać za pomocą skrótów klawiszowych:

Ctrl+Plus / Ctrl+Minus / Ctrl+0 dla czcionki,

Ctrl+Shift+C dla kontrastu

```

window.addEventListener('keydown', (e) => {
    if (!e.ctrlKey) return;
    if (e.key === '+' || e.key === '=') { e.preventDefault(); btnInc && btnInc.click(); }
    if (e.key === '-') { e.preventDefault(); btnDec && btnDec.click(); }
    if (e.key === '0') { e.preventDefault(); btnReset && btnReset.click(); }
    if (e.shiftKey && e.key.toUpperCase() === 'C') { e.preventDefault(); btnContrast &&
    btnContrast.click(); }
})

```

Podsumowanie i wnioski

Stworzona aplikacja internetowa to kompletna witryna gotowa do działania dla agencji mieszkaniowej. Praca pozwoliła nam na praktyczne zastosowanie zdobytych umiejętności z zakresu budowy aplikacji internetowych oraz bezpieczeństwa i ochrony danych. Kluczowe wnioski płynące z realizacji projektu to:

- Bezpieczeństwo od podstaw: Zastosowanie zasady Security by Design od samego początku projektowania strony i API skutecznie eliminuje podstawowe zagrożenia takie jak SQL Injection
- Ochrona danych: Wykorzystanie funkcji haszujących gwarantuje, że dane wrażliwe pozostają bezpieczne nawet w przypadku wycieku danych z bazy danych.
- Dostępność dla każdego: Dodanie rozwiązań dla niepełnosprawnych to niezbędna rzecz we współczesnym świecie tak aby każdy mógł komfortowo korzystać ze strony.